

Braillix - Electronics Plan

Written 2026-08-01. This is the electronics source of truth. Scope: **electronics only**. Mechanical/CAD issues are *flagged* here, never fixed here.

Written for a CS student with basic soldering. Every "why" is answered, because several older documents in this repo tell you to build things you do not need.

PART 0 - The short version

Question	Answer
Do we need a custom PCB?	No. Not for one cell, not for two, not for multi-cell.
Do we need custom ICs?	No. The ATmega328P plan was over-engineering.
What do I own that works?	ESP32, 28BYJ-48 motor, ULN2003 driver, hall module, 5V adapter, multimeter . That is a complete single cell.
Is the jack polarity safe?	[OK] YES - measured and confirmed correct, 2026-08-01.
Do I own a soldering iron?	[X] No. Buying guide in Part 9. Do not buy a cheap plain iron.
Can two cells work?	[OK] Yes, direct-drive, no expander - the ESP32 has the pins. See Part 10.
Does it scale past two?	[OK] Yes - 8 cells on one bus, 64 with a mux , at ~\$80/cell and zero custom parts. Part 5.
What breaks first at scale?	Power, ~11 cells - and only if they all move at once. Refresh sequentially and it goes away.
How do I stop joints breaking?	Heat-shrink + anchor the wire , not glue on the joint. Hot glue, not epoxy - the pin map will still change. Part 12.
What is the real blocker?	Nothing electronic. The circuit has never been built on a breadboard - and that needs no soldering at all.

PART 1 - Why there is no custom PCB, and never was a good reason for one

The old plan called for a "muscle board": a custom PCB carrying an **ATmega328P** per cell, talking to the ESP32 over I2C.

Kill it. Three independent reasons:

1 You cannot build it. ATmega328P-AU is **TQFP-32 at 0.8mm pin pitch**. That is a reflow-oven or hot-air part. The project's own docs already concede it must be ordered pre-assembled - which directly contradicts doing everything yourselves.

1 It solves a problem you do not have. Its job was to let many cells share a bus. You have **one** cell. A single ESP32 drives it directly, today, with parts already in hand.

1 When you *do* want many cells, an off-the-shelf module does the same job. See Part 5.

Seven documents still reference the muscle board / KiCad / ATmega: DEMO_VS_PRODUCT.md, ELECTRONICS_BOM.md, MASTER_BOM.md, PRINT_CHECKLIST.md, SHOPPING_LIST.md, SOFTWARE_TEAM_README.md, WIRING_AND_ASSEMBLY.md. Treat every one of those passages as **stale**. This file supersedes them.

"Why did we need any of this at all?"

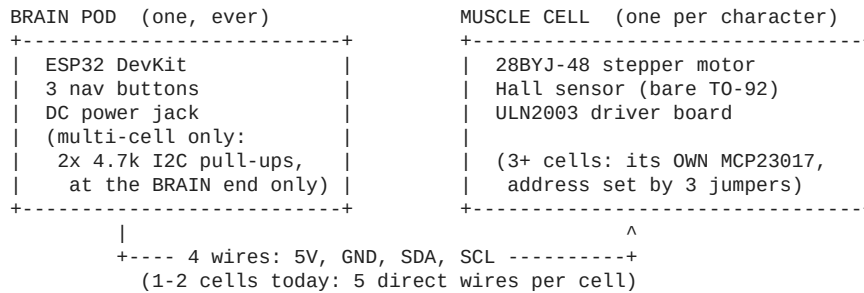
Walk the chain once and it stops feeling arbitrary:

An ESP32 pin can supply about 20mA at 3.3V.
A 28BYJ-48 motor coil wants about 200mA at 5V.
-> connect them directly and you destroy the ESP32 pin.

So you need something that takes a small signal and switches a big current.
That is ALL the ULN2003 is. Seven electronic switches in one chip.
You already own it, on the little blue board with four red LEDs.

That is the entire reason any driver exists. Nothing custom is involved.

PART 2 - What lives where



Brain pod = decides *what* to show. **Muscle cell** = makes it physically happen. **There is only ever one brain.** Cells are identical, interchangeable and addressed - Part 5 shows how that holds up to 8, 16 or 64 of them without a single custom part.

PART 3 - [!!] Pin conflict, present in the code today

```
ASSEMBLY_BIBLE.md      GPIO21 -> ULN2003 IN3      GPIO22 -> ULN2003 IN4
SOFTWARE_TEAM_README    GPIO21 = I2C SDA          GPIO22 = I2C SCL
breadboard_test.ino     #define IN3 21          #define IN4 22
```

The motor and the I2C bus are assigned **the same two pins**.

- **Single cell:** harmless. No I2C exists yet. Do not change anything mid-demo.
- **The moment a second cell appears:** they collide, and neither works.

Fix when multi-cell starts - move the motor off the I2C pins:

Signal	Now	Change to	Why
IN1	18	18	fine
IN2	19	19	fine
IN3	21	23	frees SDA
IN4	22	27	frees SCL
Hall AO	34	34	ADC1, input-only, fine

[!] **Not GPIO 5.** An earlier revision of this table said IN4 -> 5. GPIO5 is a **strapping pin** sampled at boot; a motor coil on it can stop the ESP32 starting. **27** is the correct target and is what Part 10's pin map uses.

Requires editing `breadboard_test.ino` and re-flashing. **Do not do this before the demo.**

PART 4 - Space: exactly one real constraint

Location	Space	Verdict
Cell electronics pocket	36 x 46 x 16mm	[!] The only tight spot
Pod interior	~38mm spare headroom	Fine, wildly oversized

The problem: a ULN2003 board with Dupont jumpers plugged in vertically stands ~20mm. The pocket is 16mm. It does not fit.

The fix: cut the Dupont ends off and solder the wires **flat** against the board. That drops it to ~12mm, leaving 4mm spare. This is why soldering appears at all.

Also flagged (CAD, not fixed here): the *whole blue hall module* does not fit in the base plate. Only the bare 3-legged sensor does, desoldered from the module, with three wires run back. **Not needed for the breadboard demo.**

PART 5 - Scaling to N cells

You will build two. The architecture has to claim many more, and the claim has to be true. This part is what makes it true - and the honest answer is that it costs one ?80 chip per brick and nothing else.

The problem in one line

Each cell needs **4 motor lines + 1 sensor line = 5 wires back to the brain**. Five wires per cell is a dead end: the ESP32 runs out of pins at ~4 cells, and no snap-together brick can carry a cable that grows every time you add one.

The fix: make the inter-brick cable a fixed width

DEAD END - wires grow with cell count <pre> +-----+ BRAIN === 5 ===[C1] === 5 ===[C2] === 5 ===[C3] ... +-----+ 5N wires, N <= 4 </pre>	SCALES - cable never changes <pre> +-----+ BRAIN +-----+ 4 wires: 5V GND SDA SCL [C1]--[C2]--[C3]--[C4]-- ... daisy chain each brick taps the same 4 </pre>
---	---

Every brick hangs off the **same four wires** and is told apart by an **address**, not by its own pins. Cell 9 wires exactly like cell 1. **That is the whole scaling story.**

? The decision that makes this real: one expander PER BRICK

An MCP23017 has 16 I/O. A cell needs 5. You *could* share one expander across 3 cells - but then a brick is not self-contained, and "add a cell" means rewiring its neighbours.

Give every brick its own expander instead. Wasteful on paper (5 of 16 pins used), correct in practice:

	Shared expander (3 cells each)	One per brick [OK]
Bricks identical?	No	Yes - every cell is the same object
Adding a cell	rewire neighbours	plug in, set 3 jumpers
Max cells / bus	24	8
Cost per cell	~?27	~?80

?53 per cell buys a genuinely modular brick. Take it.

Address = position. The MCP23017's A2/A1/A0 pins select one of 8 addresses (0x20-0x27). Three jumpers or a tiny DIP switch on each brick. Firmware scans the bus at boot and **discovers how many cells are attached** - no recompile to change the display length.

Use MCP23017. Do NOT use PCF8574.

	MCP23017	PCF8574
I/O	16	8
Output type	true push-pull	quasi-bidirectional, weak high side
Drives a ULN2003 input?	Yes	[!] Marginal - avoid
Package	DIP-28, hand-solderable	DIP-16
Addresses	8 (0x20-0x27)	8

Why this matters: a ULN2003 input needs a couple of mA *sourced into* it. The PCF8574's high state is a weak pull-up of roughly 100µA. It looks like it should work, and it will half-work - the worst possible failure mode. MCP23017 sources properly.

The ceiling, stated honestly

Cells	What you need	Extra cost
1-2	Direct GPIO. No expander at all. *(Part 10)*	?0
3-8	1x MCP23017 per brick, one I2C bus	~?80/cell
9-16	Second I2C bus - the ESP32 has two controllers	?0 more

Cells	What you need	Extra cost
17-64	TCA9548A I2C multiplexer, 8 buses	~?120 once

Nothing custom appears at any tier. No PCB, no ATmega, no SMD.

What actually breaks first - and it isn't addressing

[!] **Power, at about 11 cells** - but only if you move them all at once:

```
5V 3A supply = 3000mA
- ESP32      250mA
= 2750mA / 250mA per energised motor = 11 motors simultaneously
```

Refresh the cells one at a time and this limit disappears entirely - a cell draws current only while it is turning, ~0.4s. A 20-cell line refreshing sequentially never exceeds two motors' worth. This is also why *"de-energise the coils when idle"* is an architectural rule, not a nicety: hold all 20 energised and you need a 60W supply to display static text.

Three rules that must be locked in now

These are cheap to honour today and expensive to retrofit:

1 [!!] **I2C pull-ups live at the BRAIN end only - 2x 4.7k?, once, ever.**

Putting them in every brick is the classic mistake: 8 bricks x 4.7k in parallel = **590?**, which exceeds what an I2C device can sink. The bus stops working and the cause is invisible.

1 **Hall sensors move from AO to DO.** An expander has **no ADC**. Single-cell uses the analog

pin; multi-cell must use the module's digital output and its trimmer pot has to be **calibrated per cell**. Budget for that - it's a knob-turning session, not a redesign.

1 **Keep the bus short and slow-able.** I2C tolerates ~400pF total. Short jumpers across 8

bricks are fine; if a long chain goes flaky, drop the clock to 100kHz before suspecting anything else.

What this means for the two boards you're building

Build the breadboard demo on **direct GPIO** - it's unblocked today and needs no expander. Then build **both muscle boards with the expander architecture anyway**. The two cells you physically make are then not a two-cell hack; they are cells 1 and 2 of an N-cell bus, and adding cell 3 is a purchase, not a redesign. **That is what makes the claim demonstrable to the panel** - you can point at the address jumpers.

Total custom silicon required: zero. Total SMD soldering: zero.

PART 6 - What to actually do, in order

1. [OK] Jack polarity - DONE, 2026-08-01

Measured with a multimeter and **confirmed correct: red = positive, black = ground**. The single largest risk to the ESP32 is now retired. Mark the positive wire physically (nail polish, marker, a tag) so it can never be confused later.

2. Buy a soldering iron - see Part 9

You do **not** own one. Nothing in the build is blocked by this yet, because the breadboard demo needs no soldering. **Part 9 explains exactly what to buy and why the ?200 irons are a trap.**

3. Build the breadboard circuit - the actual blocker

No soldering. Dupont jumpers only. Full wiring in docs/ASSEMBLY_BIBLE.md.

```
[5V adapter] --+--> breadboard 5V rail --> ULN2003 (+)
               +--> breadboard GND rail --> ULN2003 (-)

ESP32 (powered by USB from the laptop)
GND    --> breadboard GND rail    <-- COMMON GROUND. Without it the hall reads garbage.
3V3    --> Hall VCC
GND    --> Hall GND
GPIO34 <-- Hall AO
GPIO18 --> ULN2003 IN1
GPIO19 --> ULN2003 IN2
GPIO21 --> ULN2003 IN3
```

Two rules that protect the hardware:

- 1 **Hall sensor VCC goes to 3V3, never 5V.** ESP32 GPIOs are not 5V tolerant.
- 2 **Never feed the adapter into ESP32 VIN while USB is plugged into the laptop.**

Motor from the adapter, ESP32 from USB, **grounds joined only.**

4. Flash and confirm

firmware/breadboard_test/breadboard_test.ino -> motor turns, hall finds home, dashboard appears on your phone. **That is a legitimate, demonstrable result** - and it is the same circuit the first real cell uses, so none of it is throwaway.

PART 7 - Known electronics defect, not yet fixed

breadboard_test.ino targets the wrong step position.

```
long targetStep = (long)pos * STEPS_PER_REV / 64; // lands mid-RAMP
long targetStep = (long)pos * STEPS_PER_REV / 64 + 32; // lands mid-DWELL
```

The cam's ramps are centred on slice boundaries, so pos x 64 stops the follower **on the slope** instead of on the flat. Verified numerically: at position 0 all six dots sit at factor 0.5 - half-raised, unreadable.

Not applied yet because it is tied to a mechanical re-derivation happening in the CAD fork. Recorded here so that if the bench shows half-raised dots, nobody hunts for a mechanical cause that does not exist.

PART 8 - Shopping, electronics only

[OK] **Already owned:** multimeter, ESP32, motor, ULN2003, hall module, 5V adapter, DC pigtail.

Item	Why	?
Soldering station (see Part 9)	[!] The one real gap. Temperature-controlled, not a plain iron.	~1,300
Solder, 0.8mm 60/40 leaded, rosin core	Every joint in this project	~150
Breadboard	The entire demo runs on it	~100
Wire strippers	Prepping wire ends	~200
USB-C data cable	To flash the ESP32. Charge-only cables look identical and fail silently - test yours first.	~150
Heat-shrink assortment	Insulating every splice	~100
Desoldering wick	Fixing bridges; lifting the hall sensor off its module	~80
Helping-hands / third hand	Not optional in practice - you cannot hold iron, solder and two wires with two hands	~200

~?2,280, or ~?2,130 if your existing USB-C cable carries data.

Not needed: any driver IC (you own it), resistors, capacitors, flyback diodes (inside the ULN2003), level shifters, crystals, and **any custom PCB.**

PART 9 - Buying a soldering iron, explained

You said the options confused you. They are confusing, and **most of what an electronics shop will sell you for ?200 is close to useless for this work.** Here is the whole decision.

The only choice that really matters

Buy a temperature-CONTROLLED soldering STATION. Not a plain "pencil" iron.

	Plain pencil iron ?150-300	Soldering station ?1,200-1,800
Temperature	Uncontrolled - climbs until it stops	You set it. It holds it.

	Plain pencil iron ?150-300	Soldering station ?1,200-1,800
In practice	Too hot: burns flux instantly, oxidises the tip black, lifts pads	Correct heat, joints flow properly
Tip life	Days	Months
Stand	Usually none	Included
Beginner result	Blames themselves for "being bad at soldering"	Actually learns

Why uncontrolled irons ruin beginners: solder needs the *joint* at ~250degC. A plain iron free-runs to 400degC+, which burns the flux off before it can clean the metal. Solder then balls up and refuses to stick - and it looks exactly like bad technique. It isn't. It's the tool.

This is the single highest-value ?1,000 in the whole project.

What to search for

"60W soldering station temperature controlled 936"

The 936 design is an old Hakko layout that everyone clones. Common brands in India: **Soldron, Yihua, Aptech**. They are all much the same. Roughly **?1,200-1,800**.

Reading the spec sheet

Spec	Get	Why
Temperature control	Yes - non-negotiable	The entire reason to buy a station
Wattage	60W	Not "how hot" - how *fast it recovers* after touching cold metal. 60W recovers quickly.
Heating element	Ceramic	Faster and more accurate than nichrome
Tip shape	Chisel ~2.4mm ("D24"/"D-type")	More contact area = faster heat transfer. A fine conical tip is the classic beginner mistake - it looks precise and transfers heat poorly.
Temp range	200-450degC	You'll use 320-350degC
Stand + sponge	Included	Ignore, buy brass wool if you can

Settings for this project

Solder 0.8mm 60/40 LEADED, rosin core <- leaded, deliberately
Temperature 330 C for everything here
Tip 2.4mm chisel

Use leaded 60/40, not lead-free. Lead-free melts ~40degC hotter, wets far worse, and is genuinely harder for a first-timer. Leaded is legal for hobby/education use. **Wash your hands afterwards, don't eat at the bench**, and that is the whole safety story.

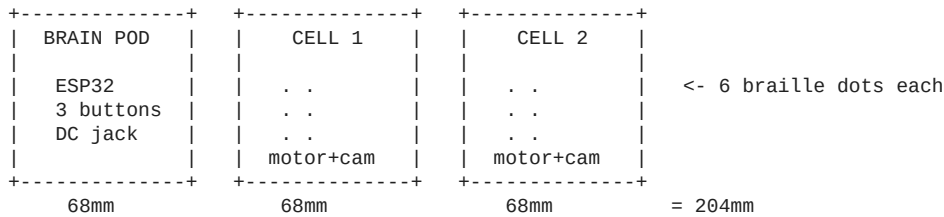
Ventilate. The smoke is burning flux, not lead - it is an irritant, not a heavy-metal vapour. Open a window; don't hunch over it.

What NOT to buy

- [X] **A plain 25W/35W pencil iron.** The trap this section exists to prevent.
- [X] **A "soldering gun"** (pistol-shaped, instant heat) - for wiring, far too crude here.
- [X] **TS100 / Pinecil USB-C smart irons** (?3,000-5,000) - genuinely excellent, but they need a good USB-C PD supply and cost triple. Overkill for this build.
- [X] **Fine conical tips.** Get the chisel.

PART 10 - The two-cell product

What it looks like



Shows TWO characters at once. Magnets hold the bricks together.
Wires run between them through the existing wire-exit holes.

Two cells = two characters. With the nav buttons you scroll a longer word through them.

? The key decision: no expander, no I2C, no extra parts

Part 5 describes an MCP23017 expander per brick. **That earns its place from 3 cells up. At two cells it is unnecessary** - the ESP32 has enough pins to drive both directly:

```
safe output pins available : 15
two cells + nav need       : 11
spare                      : 4
```

So **cell 2 costs you nothing but wire**. No new chips, no I2C, no addresses, no pull-ups.

Pin map - designed so adding cell 2 never touches cell 1

Signal	GPIO	Notes
Cell 1 IN1 / IN2 / IN3 / IN4	18, 19, 23, 27	21/22 freed - see the Part 3 conflict
Cell 1 hall AO	34	input-only, ADC1
Cell 2 IN1 / IN2 / IN3 / IN4	13, 14, 26, 33	all safe, no strapping pins
Cell 2 hall AO	35	input-only, ADC1
Nav Prev / Select / Next	32, 25, 17	INPUT_PULLUP, active-LOW
(reserved) I2C SDA / SCL	21, 22	kept free for a 3rd+ cell later

[!] **Avoid GPIO 0, 2, 5, 12, 15** entirely - they are **strapping pins** read at boot. A motor coil wired to one can stop the ESP32 booting. That is why cell 1 moves off 21/22 to 23/27.

Power

```
5V 3A adapter
|
+--> ESP32          ~250mA peak (WiFi)
+--> Cell 1 motor   ~250mA while energised
+--> Cell 2 motor   ~250mA while energised
-----
~750mA of 3000mA available  -> plenty
```

[!] **De-energise the motors when idle.** A stepper holding position draws full current and gets warm for no benefit. The firmware should release the coils after each move.

PART 11 - Soldering, from zero

Written assuming you have never made a reliable joint. Nothing here needs a steady surgeon's hand - soldering is about **heat and cleanliness**, not dexterity.

The one idea that makes soldering work

Heat the JOINT. Let the JOINT melt the solder. **Never melt solder on the iron and carry it over.**

Solder flows **toward heat** and sticks only to metal that is hot enough and chemically clean. Melting solder on the tip and dabbing it on gives you a **cold joint** - it looks attached, conducts badly, and fails weeks later. This is the mistake essentially every beginner makes.

RIGHT

1. tip touches BOTH parts
2. wait ~2 seconds
3. feed solder INTO the joint
4. it flows and wets instantly
5. remove solder, then iron

WRONG

1. melt blob on tip
2. dab blob onto parts
3. it sits there like grey chewing gum

A good joint is shiny and concave, like a tiny volcano hugging the wire. **A bad joint is dull, round and blobby** - it beaded up instead of wetting.

Before your first joint

- 1 **Set 330degC.** Wait for it to reach temperature.
- 2 **Tin the tip:** melt a little solder onto it, wipe on brass wool or damp sponge. It should be mirror-silver. A black tip transfers almost no heat - re-tin it.
- 1 **Re-tin every few joints, and always before switching off.** A tip left bare oxidises and dies.

Job 1 - DC jack to header pins *(do this first, it's the easiest)*

Your jack is an **inline pigtail**: a barrel socket with two bare wire ends. Bare wire does not sit in a breadboard.

```
[barrel socket]===red wire====> solder to pin 1 --+
                    ===black wire====> solder to pin 2 --+--> 2-pin male header
                                                    plugs into breadboard
```

- 1 Strip ~4mm of insulation from each wire.
- 2 **Tin each wire:** heat it, feed solder in, until the strands turn silver and hold together.
- 3 **Tin the two header pins** the same way.
- 4 Hold the wire against the pin, touch the iron to both, and they fuse in about a second - both surfaces are already tinned. This is called a **tack joint**.

- 1 **Slide heat-shrink over each joint before you solder the second one.** Everyone forgets. Shrink it with the side of the iron barrel or a lighter held well away.

- 1 **[!] Keep red and black clearly distinguishable.** Polarity is confirmed correct - do not lose that information at the connector.

Nothing is powered while you do this. Ideal first job.

Job 2 - ULN2003 wires flat *(only when you assemble cell 1)*

Why: with Dupont jumpers plugged in vertically the board is ~20mm tall. The pocket is **16mm**. Soldered flat it is ~12mm.

- 1 Cut the Dupont connector off six wires - IN1, IN2, IN3, IN4, +, ?.
- 2 Strip and tin each.
- 3 Feed each wire through its hole from the **top**, bend it flat along the board, solder underneath.
- 4 Snip the excess.
- 5 **[!] Check for bridges** - solder accidentally joining two adjacent pads. Hold it to the light.

A bridge between IN1 and IN2 means the motor will buzz and not turn. Fix with desoldering wick.

Job 3 - Hall sensor off its module *(cell assembly, later)*

The blue module does not fit inside the machine. Only the little black 3-legged sensor does.

- 1 Note **which leg is which** before removing anything. Photograph it.
- 2 Heat each leg and lift with tweezers, or use desoldering wick to clear the pads.
- 3 Solder three wires to the legs, **heat-shrink each individually** - they are ~1.27mm apart and a short between VCC and GND is a dead sensor.

- 1 **This is the fiddliest job in the project.** Leave it until you're comfortable.

*(Where it physically mounts is a **CAD** question - flagged, not solved here.)*

Job 4 - Adding cell 2

Cell 1 stays untouched. All you do is add wires.

The power spine - one source feeding three places:



- 1 Strip a 10mm window in the middle of a wire (don't cut it) and wrap the branch wire through it.
- 2 Solder, then heat-shrink. This is a **Western Union splice** and it is far stronger than twisting.

- 1 Alternative if you'd rather not splice: a **2-way screw terminal block**, ~?20, no soldering.
- 2 Run cell 2's four control wires to GPIO **13, 14, 26, 33** and its hall to **35**.

Firmware: add a second `AccelStepper` object on the new pins. The braille encoding, homing and dashboard all work unchanged.

When it goes wrong

Symptom	Cause	Fix
Solder balls up, won't stick	Joint not hot enough, or tip oxidised	Re-tin the tip; heat the joint longer
Joint dull and lumpy	Cold joint - moved before it set	Reheat until it flows shiny
Iron won't melt solder	Black oxidised tip	Re-tin. If it won't take solder, replace the tip
Motor buzzes, doesn't turn	Solder bridge between IN pins, or two coil wires swapped	Inspect for bridges; try swapping IN2/IN3
Wire pulls straight out	Cold joint	Redo it. A good joint won't pull off.
Everything worked, now nothing	Check the common ground first	It is almost always the ground

Practice first

Do not let the ESP32 be your first ever joint. Solder a few scrap wires together, cut them open, look at the cross-section. Twenty minutes of practice with ?5 of wire saves a ?350 board.

PART 12 - Making joints permanent (what to put on after soldering)

First, the thing almost everyone gets wrong

Solder joints don't fail because the solder breaks. They fail because the **WIRE** flexes right where it enters the joint. The solder is stiff; the wire is soft. Every wiggle concentrates at that exact boundary until the copper work-hardens and snaps - often **inside** the insulation, so it looks perfect and reads open on the multimeter.

[!] **So gluing the joint itself doesn't fix it.** A hard blob only moves the flex point 2mm along and can make it **worse**. **You have to stop the wire from moving** - anchor it 5-10mm away so the joint never sees the force. That's what "strain relief" means, and it does more than any adhesive.

Do these in order

Order	What	Why
1	Heat-shrink tubing over every joint	* Insulates *and* stiffens the transition. Slide it on before you solder - everyone forgets this once.
2	Anchor the cable - zip tie, clamp, or a dab of hot glue on the wire <i>*away*</i> from the joint	This is the actual fix. The joint stops flexing.
3	<i>*Only then*</i> consider a coating	Everything below is a bonus, not a substitute

The coatings, honestly compared

Option	Hardness	Reworkable?	Verdict for Brailix
Hot glue	soft, rubbery	[OK] peels off	* Use this. Cheap, instant, great strain relief, and you *will* need to undo things. Softens ~70degC - irrelevant here.
Neutral-cure RTV silicone	flexible	[OK] mostly	* Best vibration resistance. [!] Must be neutral-cure - see warning below.
2-part epoxy	rock hard	[X] never	Only for the final, fully-tested build. Hard *and brittle* - it transfers shock to the joint instead of absorbing it.
Conformal coating (acrylic spray)	thin film	[OK] with thinner	Protects against moisture/oxidation. Adds no mechanical strength. Not needed indoors.
UV resin (you have a resin printer)	hard, brittle	[X]	Tempting since it's on hand - but it won't cure in shadow under a wire, leaving sticky uncured resin. Skip it.
Nail polish	thin	[OK] acetone	Genuinely fine for locking a *screw* or marking polarity. Not a structural adhesive.

[!!!] Two warnings worth more than the rest of this section

1 **Never use acetic-cure silicone** (the hardware-store bathroom kind that smells of vinegar). It releases **acetic acid** as it cures, which corrodes copper and eats your joints over months. The tube must say **neutral cure** / oxime / alkoxy.

1 **Do not epoxy anything until the whole circuit is verified working.** Epoxy on ESP32 header pins means the board is scrap if the pin map changes - and *your pin map is going to change* (Part 3's conflict fix, and the expander migration in Part 5). **Hot glue until the design is frozen.**

Where each one belongs on this build

```
DC jack wires -----> heat-shrink + hot glue blob at the strain point
ULN2003 flat wires ----> heat-shrink; hot glue ONLY after the pocket fit is confirmed
Hall sensor 3 wires ---> heat-shrink; the TO-92 itself gets epoxied into the CAD pocket
Inter-brick cable -----> connector, not glue -- it MUST come apart (Part 5 is modular)
Nav buttons -----> hot glue, so the pod can be opened
```

[!] **Never glue the inter-brick connection.** The entire scaling argument in Part 5 depends on bricks separating. Glue there and you've built a permanent 2-cell device.

[!] **Keep every adhesive away from the cam, the linkage feet and the dot holes.** A drop of hot glue in the mechanism is far harder to remove than a bad solder joint.

The 5-second test that beats any coating

Tug it. Pull each wire firmly by hand before you insulate anything. A properly soldered joint will not come off - if it moves, no glue on earth will save it. Redo the joint.

PART 13 - Flagged for the CAD fork (do not fix here)

1 **Hall module does not fit** the base plate - needs the bare TO-92 desoldered, and a pocket that can hold it.

1 **DC jack has no thread or nut** - the pod lid cannot clamp it. Needs either a mechanical retainer in the box or a different jack.

1 **ULN2003 pocket is 16mm** - drives the solder-flat requirement. Working, but zero margin.